The exact output will depend on how your environment (assuming PHP is using its default UTF-8 encoding) handles the *0x81* byte value, but this demonstrates that we've truncated the `$name` string precisely in the middle of a multi-byte sequence. The following demonstrates exactly what has happened:

```
print bin2hex("\u{141}ukasz") . PHP_EOL;
print bin2hex(substr("\u{141}ukasz", 1)) . PHP_EOL;
```

```
Output:
c581756b61737a
81756b61737a
```

Fortunately, there are a couple of `mbstring` variants that solve this problem. `mb_substr()` operates on character offsets and lengths and is encoding-aware.

```
print mb_substr($name, -7) . PHP_EOL;
```

```
Output:
Łukasz!
```

`mb_strcut()` is halfway between `substr()` and its multi-byte equivalent in that it operates on byte values but is still encoding-aware and won't truncate multi-byte characters: If a cut would occur in the middle of a multi-byte sequence, then `mb_strcut()` will include all preceding bytes to the character boundary:

```
print bin2hex(mb_strcut("\u{141}ukasz", 1)) . PHP_EOL;
```

```
Output:
c581756b61737a
```

Despite asking `mb_strcut()` to discard byte zero, it has been included in output because it was part of the UTF-8 character *0xC5 0x81*.

strstr(), stristr(), strrchr()

`strstr()` (and its alias `strchr()`) will search a string and return the portion that matches the first occurrence; `stristr()` does the same but is case insensitive:

```
$ship = 'Registration: NCC-1701-D';
print strstr($ship, 'NCC') . PHP_EOL;
var_dump(strstr($ship, 'ncc'));
print stristr($ship, 'ncc') . PHP_EOL;
```